

What is Machine Learning?

- ML = letting the computer learn a pattern from data

How to approach an ML exercise

1. Read the question → identify target (y)
 2. Choose features (X)
 3. Pick one baseline model
 4. Fit → predict → evaluate
- We always have:
 - **Input (X)** → information
 - **Output (y)** → what we want to predict

What are X and y?

Hours studied	Attendance	Exam score
2	60%	55
5	80%	75

- **y (target)** → *what question are we answering?*

“What score will the student get?” → $y = \text{Exam score}$

- **X (features)** → information used to predict y

$X = [\text{Hours studied}, \text{Attendance}]$

y = one column you want to predict

X = everything else that is useful

Regression vs Classification

Output type:

Question	Type
Predict price / score	Regression
Predict yes/no / class	Classification

Simple decision rule:

- Number → **Regression**
- Category → **Classification**

Which model to start with?

Problem	Start with
Regression	<code>LinearRegression</code>
Classification	<code>KNN</code> or <code>LogisticRegression</code>

Example:

“Predict house prices using size and number of rooms”

Read the question properly

- What is being predicted? → **Price** - That's → y
- What's the information I know about houses? → **size, number of rooms** - That's X

Note: Never guess models before defining y .

Choose the simplest model

- Price = number → regression
- Start with `LinearRegression`

Fit → Predict → Evaluate

```
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

- Regression → error (MSE)
- Classification → accuracy

Who does what in ML

The ML workflow

Data → Prepare → Learn → Evaluate

Library	Role	Simple meaning
NumPy	Math engine	Works with numbers & arrays
Pandas	Data handling	Tables (rows & columns)
scikit-learn	Machine Learning	Models, training, evaluation

NumPy – *Numbers & mathematics*

- Arrays
- Vectors
- Mathematical operations

Used when:

- Data is purely numeric
- We want fast math
- ML models internally use NumPy

Example:

```
import numpy as np

X = np.array([[1], [2], [3]])
y = np.array([10, 20, 30])
```

sklearn models always convert data to NumPy inside.

Pandas – *Real-world data*

- Tables
- CSV, Excel, databases
- Column names

Used when:

- Data comes from files
- Data has columns
- We need cleaning

Example:

```
import pandas as pd

data = pd.read_csv("students.csv")
X = data[['hours', 'attendance']]
y = data['score']
```

Most ML projects start with Pandas.

scikit-learn – *Learning from data*

- Models
- Train/test split
- Metrics

Example:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, y)
```

scikit-learn **does NOT store data** — it learns from it.

How we decide which tool to use

Decision Table

Situation	Use
Data in CSV / Excel	Pandas
Simple numeric example	NumPy
Training ML model	scikit-learn
Small examples/ only learning purposes	Define data manually
Real project	Load data

Pandas handles data.

NumPy handles math.

scikit-learn handles learning.

THREE COMMON WAYS TO GET DATA

1: Using built-in sklearn datasets

Example: Iris dataset

```
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data
y = iris.target
```

- Data already split into X and y
- No Pandas needed
- Focus = learning ML logic

2: Creating data manually

When & why?

- only to understand concepts
- Understanding X and y
- Zero confusion

Example:

```
X = [[1],[2],[3],[4]]    # hours studied (2D)
y = [50, 60, 70, 80]    # student scores
```

3: Loading a CSV file

When & why?

- Real-world ML
- Assignments
- Projects

Example:

```
import pandas as pd

data = pd.read_csv("students.csv")

X = data[['hours', 'attendance']]
y = data['score']
```

Most professional ML uses this

Why sometimes X, y come ready-made (and sometimes not)

Case	Why
sklearn dataset	Already structured
CSV file	We must decide
Manual data	We define everything

How scikit-learn handles ALL cases

No matter how data comes in:

```
model.fit(X, y)
model.predict(X_new)
```

```
CSV / Excel → Pandas → X, y
                ↓
            scikit-learn model
                ↓
            Predictions
```

OR

```
sklearn dataset → X, y → model
```

Different examples show different real-life situations, but the ML logic is always the same.

Think of Pandas as Excel, NumPy as a calculator, and scikit-learn as the brain.